

I. TÌM HIỂU API

1. Giới thiệu về Các API của Hệ thống CRM trong Odoo

Odoo CRM sử dụng kiến trúc **modular API** dựa trên Python 3.x và JSON-RPC 2.0 (chuẩn web service), cho phép tích hợp đa kênh (app, website, IoT) mà không cần middleware. Các API được thiết kế theo mô hình **RESTful-like** (dù không hoàn toàn REST), hỗ trợ CRUD (Create, Read, Update, Delete) trên các models CRM chính:

- **crm.lead**: Quản lý leads/opportunities (tạo lead từ form đăng ký, chuyển đổi thành khách hàng).
- **res.partner**: Hồ sơ khách hàng (cập nhật RFM segment, email cho ưu đãi).
- **sale.order**: Đơn hàng (truy vấn lịch sử mua để phân tích hành vi).
- **mailing.mailing**: Chiến dịch email.

2. Các Loại API Chính trong Hệ thống CRM Odoo

Odoo CRM sử dụng các API cốt lõi của Odoo, tập trung vào models CRM. Các loại chính được phân loại theo phạm vi sử dụng:

- **External API (JSON-RPC/XML-RPC)**: Cho bên ngoài (app, third-party).
- **Internal API (ORM API)**: Cho lập trình nội bộ (module tùy chỉnh). Dùng trong Python code, hỗ trợ decorators như `@api.model`.
- **Extract API**: Cho xử lý tài liệu (AI-based, mới Odoo 19). Hỗ trợ OCR cho hóa đơn CRM.
- **Webhooks API**: Cho sự kiện realtime (event-driven). Cấu hình qua UI, gọi callback URL khi sự kiện xảy ra (lead created).

2.1. External API

2.1.1. External RPC API

Đây là API dựa trên XML-RPC và JSON-RPC, hiện đang được sử dụng nhưng sẽ bị loại bỏ trong Odoo 20 (dự kiến 2026). JSON-2 API là giải pháp thay thế.

a. Đặc điểm của XML-RPC và JSON-RPC:

- Endpoint:
 - XML-RPC: `/xmlrpc`, `/xmlrpc/2`
 - JSON-RPC: `/jsonrpc`

- **Không trạng thái:** Mỗi yêu cầu phải cung cấp thông tin xác thực (database, user ID, password hoặc API key).
- **Hỗ trợ ngôn ngữ:** Có thể truy cập từ nhiều ngôn ngữ lập trình như Python, PHP, Java, v.v., miễn là có thư viện hỗ trợ XML-RPC hoặc JSON-RPC.
- **Lưu ý về PHP:** Kể từ PHP8, phần mở rộng XML-RPC không được cài đặt mặc định, cần tham khảo hướng dẫn cài đặt.

b. Xác thực:

- Quy trình:
 - Gửi yêu cầu tới endpoint `/xmlrpc/2/common` hoặc `/jsonrpc` với phương thức `authenticate`.
 - Cung cấp: tên cơ sở dữ liệu, tên người dùng, mật khẩu (hoặc API key), và một object rỗng.
 - Kết quả trả về là **user ID** của người dùng nếu xác thực thành công.
- **API Key:** Có thể sử dụng API key thay cho mật khẩu. API key được tạo tương tự như trong JSON-2 API.
- **Odoo Online:** Người dùng không có mật khẩu cục bộ. Cần đặt mật khẩu cho tài khoản qua *Settings* → *Users & Companies* → *Users* → *Action* → *Change Password*.

c. Các chức năng chính:

XML-RPC và JSON-RPC cho phép gọi các phương thức của mô hình Odoo thông qua hàm `execute_kw`. Các thao tác phổ biến bao gồm:

- **List records:** Tìm kiếm bản ghi với `search`:

```
models.execute_kw(db, uid, password, 'res.partner', 'search',
[[['is_company', '=', True]], {'limit': 10}])
```

Trả về danh sách ID của các bản ghi phù hợp.

- **Count records:** Đếm bản ghi với `search_count`:

```
models.execute_kw(db, uid, password, 'res.partner', 'search_count',
[[['is_company', '=', True]]])
```

- **Read records:** Đọc dữ liệu bản ghi với *read*:

```
models.execute_kw(db, uid, password, 'res.partner', 'read', [ids],  
{ 'fields': ['name', 'email'] })
```

- **List record fields:** Lấy thông tin trường của mô hình với *fields_get*:

```
models.execute_kw(db, uid, password, 'res.partner', 'fields_get', [],  
{ 'attributes': ['string', 'type'] })
```

- **Search and read:** Kết hợp tìm kiếm và đọc với *search_read*:

```
models.execute_kw(db, uid, password, 'res.partner', 'search_read',  
[[['is_company', '=', True]]], { 'fields': ['name', 'email'], 'limit': 10 })
```

- **Create records:** Tạo bản ghi mới với *create*:

```
models.execute_kw(db, uid, password, 'res.partner', 'create', [{ 'name':  
'New Partner', 'email': 'new@partner.com' }])
```

- **Update records:** Cập nhật bản ghi với *write*:

```
models.execute_kw(db, uid, password, 'res.partner', 'write', [[id],  
{ 'name': 'Updated Partner' }])
```

- **Delete records:** Xóa bản ghi với *unlink*:

```
models.execute_kw(db, uid, password, 'res.partner', 'unlink', [[id]])
```

- **Inspection and introspection:** Kiểm tra quyền truy cập hoặc lấy thông tin mô hình:

```
models.execute_kw(db, uid, password, 'res.partner', 'check_access_rights',  
['read'], { 'raise_exception': False })
```

d. Ví dụ sử dụng:

Dưới đây là một ví dụ sử dụng XML-RPC để tìm kiếm và đọc bản ghi trong mô hình *res.partner*:

```

from xmlrpc.client import ServerProxy

# Thông tin kết nối
url = 'https://mycompany.odoo.com'
db = 'mycompany'
username = 'admin'
password = 'your_password'

# Kết nối tới endpoint common
common = ServerProxy(f'{url}/xmlrpc/2/common')
uid = common.authenticate(db, username, password, {})

# Kết nối tới endpoint object
models = ServerProxy(f'{url}/xmlrpc/2/object')

# Tìm kiếm và đọc bản ghi
partners = models.execute_kw(db, uid, password, 'res.partner',
                             'search_read',
                             [[['is_company', '=', True]]],
                             {'fields': ['name', 'email'], 'limit': 10})
print(partners)

```

e. Lợi ích và hạn chế:

- Lợi ích:
 - Hỗ trợ rộng rãi cho nhiều ngôn ngữ lập trình.
 - Cho phép thực hiện hầu hết các thao tác trên mô hình Odoo.
- Hạn chế:
 - Sẽ bị ngừng hỗ trợ trong Odoo 20 (2026).
 - Tài liệu không đầy đủ, thiếu ví dụ ngôn ngữ-agnostic.
 - Khó khăn trong việc khám phá các phương thức và trường có sẵn nếu không sử dụng công cụ như `/doc` endpoint.

2.1.2. External API (JSON-2 API)

API mới, sử dụng giao thức HTTP qua endpoint `/json/2`, được giới thiệu trong Odoo 19. Đây là giải pháp hiện đại hơn, dễ sử dụng hơn và sẽ thay thế các API RPC cũ.

a. Đặc điểm của JSON-2 API:

- **Endpoint chính:** API hoạt động qua endpoint `/json/2` và sử dụng giao thức **HTTP**.
- **Không trạng thái (Stateless):** Mỗi yêu cầu API phải cung cấp đầy đủ thông tin xác thực (database, API key, hoặc thông tin người dùng).

- **Bảo mật:** JSON-2 API sử dụng mô hình bảo mật tiêu chuẩn của Odoo, kiểm tra quyền truy cập và quy tắc bản ghi dựa trên người dùng thực hiện yêu cầu.
- **Khác biệt với RPC:** Không yêu cầu user ID và password như XML-RPC/JSON-RPC. Thay vào đó, nó sử dụng **API key** hoặc lấy thông tin người dùng từ ngữ cảnh (context).

b. Xác thực:

- **API Key:** Để sử dụng JSON-2 API, bạn cần tạo một API key trong Odoo:
 - Đăng nhập với tài khoản quản trị.
 - Vào Settings → Users & Companies → Users.
 - Chọn người dùng, vào tab *Security*, và tạo *API Key*.
 - API key này sẽ được sử dụng thay cho mật khẩu trong các yêu cầu API.
- **Lấy User ID:** Nếu cần user ID, bạn có thể gửi yêu cầu tới *res.users/context_get* với endpoint */json/2* mà không cần cung cấp ID (hệ thống sẽ tự động lấy ID người dùng hiện tại từ API key).
- **Bot Users:** Để tích hợp tự động, nên tạo tài khoản bot với quyền tối thiểu và API key riêng, đồng thời vô hiệu hóa xác thực bằng mật khẩu để giảm nguy cơ bị xâm phạm.

c. Các chức năng chính:

JSON-2 API cho phép truy cập vào tất cả các **phương thức công khai (public methods)** của mô hình Odoo, bao gồm:

- **ORM Methods:** Các phương thức của Object-Relational Mapping như *create*, *write*, *unlink*, *search*, *read*,...
- **Custom Methods:** Các phương thức tùy chỉnh được định nghĩa trong các module Odoo.

Ví dụ, chúng ta có thể:

- Tạo bản ghi mới trong mô hình *res.partner* (đối tác).
- Đọc dữ liệu từ mô hình *product.product* (sản phẩm).
- Gọi phương thức tùy chỉnh của một module cụ thể.

d. Các endpoint liên quan:

Ngoài */json/2*, JSON-2 API cũng sử dụng các **controller** liên quan đến quản lý cơ sở dữ liệu, hoạt động qua endpoint */web/database*. Các controller này sử dụng **HTTP POST**:

- **/web/database/create:** Tạo cơ sở dữ liệu mới. Yêu cầu các tham số như *master_pwd* (mật khẩu chính), *name* (tên cơ sở dữ liệu), *login*, *password*, *demo* (dữ liệu demo), *lang* (ngôn ngữ), và *phone*.
- **/web/database/duplicate:** Sao chép cơ sở dữ liệu. Yêu cầu *master_pwd*, *name* (tên cơ sở dữ liệu gốc), *new_name* (tên cơ sở dữ liệu mới), và *neutralize_database* (tùy chọn, mặc định là không vô hiệu hóa).
- **/web/database/list:** Liệt kê các cơ sở dữ liệu. Yêu cầu một JSON object rỗng và trả về danh sách cơ sở dữ liệu trong *result*.
- **/web/version:** Lấy phiên bản server.
- **res.lang** và **res.country:** Lấy danh sách ngôn ngữ và quốc gia qua JSON-2 API.
- **migrate_databases:** Chưa có API lập trình, không được hỗ trợ.

e. Công cụ hỗ trợ:

- **/doc endpoint:** Trong Odoo 19, endpoint */doc* tự động tạo tài liệu về các mô hình, trường, và phương thức có sẵn cho một cơ sở dữ liệu cụ thể, giúp lập trình viên khám phá API dễ dàng hơn.
- **Odoo Language Server:** Công cụ này cung cấp tính năng tự động hoàn thành, kiểm tra file, và điều hướng mã nguồn cho các IDE như VS Code, PyCharm, Vim, và Zed.

f. Lợi ích và hạn chế:

- Lợi ích:
 - Dễ sử dụng hơn so với XML-RPC/JSON-RPC.
 - Hỗ trợ tài liệu tự động qua */doc*.
 - Tích hợp tốt với các hệ thống bên ngoài nhờ giao thức HTTP.
- Hạn chế:
 - Yêu cầu gói trả phí Custom.
 - Tài liệu vẫn đang được cải thiện, có thể thiếu ví dụ cụ thể.
 - Không hỗ trợ Swagger/OpenAPI trực tiếp cho JSON-RPC.

2.2. Internal API

2.2.1. Tổng quan về Internal API (ORM API) trong Odoo

ORM API là một phần cốt lõi của Odoo, cho phép các nhà phát triển tương tác với cơ sở dữ liệu thông qua các đối tượng Python thay vì viết các câu lệnh SQL trực tiếp. ORM API cung cấp một cách trừu tượng để định nghĩa mô hình dữ liệu, truy vấn, và thao tác với bản ghi, giúp việc phát triển nhanh hơn và giảm nguy cơ lỗi.

a. Các thành phần chính của ORM API:

- **Models:** Đại diện cho các bảng trong cơ sở dữ liệu (*res.partner*, *product.product*, ...).
- **Fields:** Các cột trong bảng, định nghĩa kiểu dữ liệu và thuộc tính (*Char*, *Integer*, *Many2one*, ...).
- **Recordsets:** Tập hợp các bản ghi (records) của một mô hình, có thể là 0, 1, hoặc nhiều bản ghi.
- **Environment:** Bối cảnh (context) chứa thông tin về người dùng, cơ sở dữ liệu, và các metadata khác.
- **Methods:** Các phương thức để thao tác dữ liệu, như *create*, *write*, *search*, *unlink*.

b. Đặc điểm nổi bật:

- **Tính mô-đun:** ORM API được thiết kế để hỗ trợ kiến trúc mô-đun của Odoo, cho phép mở rộng và tùy chỉnh dễ dàng.
- **Kiểm soát quyền truy cập:** Mọi thao tác đều được kiểm tra quyền truy cập dựa trên người dùng hiện tại.
- **Hiệu suất:** ORM API sử dụng bộ nhớ đệm (cache) và các cơ chế tối ưu để giảm truy vấn cơ sở dữ liệu.

2.2.2. Nội dung chi tiết của ORM API

a. Models:

Mô hình (Model) trong Odoo tương ứng với một bảng trong cơ sở dữ liệu. Mỗi mô hình được định nghĩa bằng một lớp Python kế thừa từ *odoo.models.Model*.

Cách định nghĩa mô hình:

```
from odoo import models, fields

class MyModel(models.Model):
    _name = 'my.model' # Tên mô hình (bảng trong cơ sở dữ liệu)
    _description = 'My Custom Model' # Mô tả mô hình

    name = fields.Char(string='Name', required=True) # Trường văn bản
    value = fields.Integer(string='Value') # Trường số nguyên
```

Thuộc tính quan trọng:

- **_name:** Tên duy nhất của mô hình, sử dụng định dạng *module_name.model_name*.
- **_description:** Mô tả hiển thị trong giao diện người dùng.
- **_auto:** Nếu *False*, bảng không được tạo tự động (dùng cho mô hình tạm thời).
- **_inherit:** Kế thừa từ mô hình khác để mở rộng hoặc tùy chỉnh.

b. Fields

Trường (Field) định nghĩa các cột trong bảng và kiểu dữ liệu của chúng. Odoo hỗ trợ nhiều loại trường, từ cơ bản đến quan hệ.

Các loại trường cơ bản:

- **Char:** Chuỗi văn bản một dòng.
- **Text:** Văn bản nhiều dòng.
- **Integer:** Số nguyên.
- **Float:** Số thực.
- **Boolean:** Giá trị *True/False*.
- **Date:** Ngày (định dạng *YYYY-MM-DD*).
- **Datetime:** Ngày và giờ (định dạng *YYYY-MM-DD HH:MM:SS*).
- **Binary:** Dữ liệu nhị phân (ví dụ: tệp, hình ảnh).
- **Html:** Nội dung HTML.

Các loại trường quan hệ:

- **Many2one:** Quan hệ một-nhiều (ví dụ: một đối tác có một quốc gia).

```
country_id = fields.Many2one('res.country', string='Country')
```

- **One2many:** Quan hệ nhiều-một (ví dụ: một đối tác có nhiều liên hệ).

```
contact_ids = fields.One2many('res.partner', 'parent_id', string='Contacts')
```

- **Many2many:** Quan hệ nhiều-nhiều (ví dụ: một đối tác thuộc nhiều nhóm).

```
group_ids = fields.Many2many('res.groups', string='Groups')
```

Thuộc tính trường:

- **string**: Nhân hiển thị trong giao diện.
- **required**: Bắt buộc nhập.
- **default**: Giá trị mặc định.
- **domain**: Ràng buộc giá trị (ví dụ: `[('is_company', '=', True)]`).
- **ondelete**: Hành động khi bản ghi liên quan bị xóa (*set null, restrict, cascade*).
- **compute**: Phương thức tính toán giá trị (dùng cho trường computed).
- **store**: Lưu trữ trong cơ sở dữ liệu (mặc định là *True* cho trường computed).
- **check_company**: Kiểm tra tính hợp lệ của công ty liên quan.

Ví dụ về trường computed:

```
total = fields.Float(compute='_compute_total', store=True)

def _compute_total(self):
    for record in self:
        record.total = record.value * 2
```

c. Recordsets:

Recordset là tập hợp các bản ghi của một mô hình. Recordset có thể rỗng, chứa một bản ghi, hoặc nhiều bản ghi.

Cách tạo recordset:

```
# Lấy tất cả đối tác
partners = self.env['res.partner'].search([])

# Lấy một đối tác cụ thể
partner = self.env['res.partner'].browse(1)
```

Thao tác với recordset:

- **search**: Tìm kiếm bản ghi dựa trên domain.

```
partners = self.env['res.partner'].search([('is_company', '=', True)])
```

- **browse**: Lấy bản ghi theo ID.
- **create**: Tạo bản ghi mới.
- **write**: Cập nhật bản ghi.
- **unlink**: Xóa bản ghi.

d. Environment

Environment lưu trữ bối cảnh của ORM, bao gồm:

- **cr**: Con trỏ cơ sở dữ liệu (database cursor).
- **uid**: ID của người dùng hiện tại (dùng để kiểm tra quyền).
- **context**: Từ điển chứa metadata (ví dụ: ngôn ngữ, múi giờ).
- **registry**: Ánh xạ từ tên mô hình đến mô hình.
- **cache**: Bộ nhớ đệm để tối ưu hiệu suất.

Cách truy cập:

```
1 env = self.env
2 user = env.user # Lấy người dùng hiện tại
3 cursor = env.cr # Lấy con trỏ cơ sở dữ liệu
```

Ví dụ sử dụng:

```
# Tìm đối tác là công ty
partners = self.env['res.partner'].search([('is_company', '=', True), ('customer', '=', True)])
```

e. Common ORM Methods:

ORM cung cấp các phương thức phổ biến để thao tác dữ liệu:

- **create(values)**: Tạo bản ghi mới.

```
self.env['res.partner'].create({'name': 'New Partner', 'email': 'new@partner.com'})
```

- **search(domain, limit=None, order=None)**: Tìm kiếm bản ghi.

```
self.env['res.partner'].search([('name', 'like', 'Partner')], limit=10)
```

- **write(values)**: Cập nhật bản ghi.

```
partners.write({'name': 'Updated Partner'})
```

- **unlink()**: Xóa bản ghi.

```
partners.unlink()
```

- **read(fields):** Đọc giá trị của các trường.

```
partners.read(['name', 'email'])
```

- **search_read(domain, fields, limit=None):** Kết hợp tìm kiếm và đọc.

```
self.env['res.partner'].search_read([('is_company', '=', True)], ['name', 'email'], limit=10)
```

- **fields_get():** Lấy thông tin trường của mô hình.

```
self.env['res.partner'].fields_get(['name', 'email'])
```

f. Relational Fields Commands:

Đối với trường quan hệ (*One2many*, *Many2many*), Odoo sử dụng các lệnh đặc biệt (commands) để thao tác. Mỗi lệnh là một tuple (*command_id*, *record_id*, *values*).

Các lệnh phổ biến:

- (0, 0, *values*): Tạo bản ghi mới và liên kết.
- (1, *id*, *values*): Cập nhật bản ghi hiện có.
- (2, *id*, 0): Xóa bản ghi.
- (3, *id*, 0): Ngắt liên kết bản ghi (không xóa).
- (4, *id*, 0): Liên kết bản ghi hiện có.
- (5, 0, 0): Xóa tất cả liên kết.
- (6, 0, *ids*): Thay thế danh sách liên kết bằng danh sách mới.

Ví dụ:

```
partner = self.env['res.partner'].browse(1)
partner.write({
    'contact_ids': [
        (0, 0, {'name': 'Contact 1', 'email': 'contact1@example.com'}),
        (4, existing_contact_id, 0)
    ]
})
```

g. Inheritance and Extension:

Odoo hỗ trợ kế thừa để mở rộng mô hình hiện có:

- **_inherit:** Kế thừa để thêm trường hoặc phương thức.

```
class CustomPartner(models.Model):
    _inherit = 'res.partner'
    custom_field = fields.Char(string='Custom Field')
```

- **_inherits:** Kế thừa để nhúng mô hình khác (delegation).

```
class CustomModel(models.Model):
    _name = 'custom.model'
    _inherits = {'res.partner': 'partner_id'}
    partner_id = fields.Many2one('res.partner')
```

h. Constraints and Validation:

Odoo hỗ trợ các ràng buộc để đảm bảo tính toàn vẹn dữ liệu:

- **@api.constrains:** Ràng buộc dựa trên trường.

```
from odoo.exceptions import ValidationError

@api.constrains('value')
def _check_value(self):
    for record in self:
        if record.value < 0:
            raise ValidationError("Value must be positive")
```

- **SQL Constraints:** Ràng buộc ở mức cơ sở dữ liệu.

```
_sql_constraints = [
    ('name_unique', 'UNIQUE(name)', 'Name must be unique')
]
```

i. Performance Optimization:

- **Cache:** Environment lưu trữ bộ nhớ đệm để giảm truy vấn.
- **auto_join:** Cho phép tạo JOIN trong truy vấn để tối ưu hiệu suất (dùng cho trường quan hệ).
- **compute_sudo:** Tính toán trường với quyền superuser để bỏ qua kiểm tra quyền.
- **Profiling:** Sử dụng công cụ profiling để tìm điểm nghẽn hiệu suất.

k. Changelog (Odoo 19.0):

Một số thay đổi quan trọng trong ORM API của Odoo 19:

- **search_display_name**: Tìm kiếm theo tên sử dụng `_search_display_name` như các trường khác.
- **check_access, has_access, _filtered_access**: Các phương thức mới để kiểm tra quyền và quy tắc truy cập.
- **name_get deprecated**: Thay bằng đọc trường `display_name`.
- **SQL wrapper**: Giới thiệu đối tượng SQL để xử lý truy vấn an toàn hơn.

2.2.3. Liên kết với External API

ORM API (Internal API) là nền tảng cho **External API** (JSON-2 API, XML-RPC, JSON-RPC). Các phương thức ORM như `create`, `search`, `write` có thể được gọi từ bên ngoài thông qua:

- **JSON-2 API**: Gửi yêu cầu HTTP tới `/json/2` với tên mô hình và phương thức.
- **XML-RPC/JSON-RPC**: Sử dụng `execute_kw` để gọi các phương thức ORM.

Ví dụ, lệnh `self.env['res.partner'].create({'name': 'Test'})` trong Python có thể được gọi qua XML-RPC như sau:

```
models.execute_kw(db, uid, password, 'res.partner', 'create', [{'name': 'Test'}])
```

2.2.4. Hướng dẫn thực hiện

Để sử dụng ORM API hiệu quả, có thể làm theo các bước sau:

- Tạo mô hình mới:
 - Định nghĩa lớp kế thừa `odoo.models.Model`.
 - Thêm các trường và phương thức cần thiết.
- Thao tác dữ liệu:
 - Sử dụng `self.env['model.name']` để truy cập mô hình.
 - Áp dụng các phương thức như `search`, `create`, `write`.
- Tối ưu hiệu suất:
 - Sử dụng `search_read` thay vì `search` + `read`.
 - Kích hoạt `auto_join` cho các trường quan hệ nếu cần.
- Kiểm tra quyền:
 - Sử dụng `sudo()` để bỏ qua quyền tạm thời (cẩn thận với bảo mật).
 - Kiểm tra quyền với `check_access_rights`.
- Debugging:
 - Kích hoạt `Developer Mode` để kiểm tra mô hình và trường.
 - Sử dụng `fields_get` để khám phá cấu trúc mô hình.

2.3. Extract API

2.3.1. Tổng quan về Extract API

Extract API là một phần của hệ thống **Document Processing** trong Odoo, được cung cấp thông qua module **iap_extract** (In-App Purchase Extract). API này cho phép các ứng dụng bên ngoài gửi tài liệu (PDF hoặc hình ảnh) tới Odoo để trích xuất thông tin như số hóa đơn, ngày, tổng tiền, hoặc chi tiết nhà cung cấp. Sử dụng công nghệ OCR và AI để phân tích tài liệu và trả về dữ liệu có cấu trúc.

Đặc điểm chính:

- **Chức năng chính:** Trích xuất dữ liệu từ tài liệu (hóa đơn, biên lai, sao kê ngân hàng) và tích hợp với các mô hình Odoo như *account.move* (hóa đơn) hoặc *hr.expense* (chi phí).
- **Công nghệ:** Kết hợp OCR, AI, và học máy để nhận dạng văn bản và phân tích dữ liệu.
- **Tích hợp:** Hoạt động thông qua endpoint HTTP `/odoo/extract/<operation>`, sử dụng giao thức JSON.
- **Yêu cầu:** Cần tài khoản *In-App Purchase (IAP)* và đủ *Extract credits* để sử dụng API.
- **Ứng dụng:** Chủ yếu được sử dụng trong các module *Invoicing*, *Expenses*, *Bank Synchronization*.

2.3.2. Nội dung chi tiết

a. Tổng quan về Extract API:

Extract API hoạt động thông qua các yêu cầu HTTP POST tới endpoint `/odoo/extract/<operation>`.

Các thao tác chính bao gồm:

- Gửi tài liệu để phân tích.
- Lấy kết quả phân tích.
- Kiểm tra trạng thái tài liệu.
- Tích hợp dữ liệu trích xuất vào các bản ghi Odoo.

API sử dụng **JSON** để gửi và nhận dữ liệu, với các tham số như:

- **file:** Nội dung tài liệu (PDF hoặc hình ảnh, mã hóa base64).
- **dbuuid:** UUID của cơ sở dữ liệu Odoo.
- **api_key:** Khóa API để xác thực.
- **document_type:** Loại tài liệu (*invoice*, *expense*, *bank_statement*,...).

b. Yêu cầu sử dụng Extract API:

- Tài khoản IAP:
 - Đăng ký tài khoản IAP trong Odoo (*Settings* → *In-App Purchases*).
 - Nạp *Extract credits* để sử dụng API.
- API Key:
 - Tạo API key trong Odoo (*Settings* → *Users & Companies* → *Users* → *Security* → *API Key*).
- Cơ sở dữ liệu UUID:
 - Lấy UUID của cơ sở dữ liệu từ Odoo (*Settings* → *Technical* → *Database*).
- Module hỗ trợ:
 - Cài đặt các module như *Invoicing*, *Expenses*, *Accounting* để tích hợp dữ liệu trích xuất.

c. Các endpoint chính:

Extract API sử dụng endpoint `/odoo/extract/<operation>` với các thao tác cụ thể:

- `/odoo/extract/endpoint`
- `/odoo/extract/status`
- `/odoo/extract/validate`

d. Dữ liệu trích xuất:

Dữ liệu trả về từ Extract API phụ thuộc vào loại tài liệu:

- Hóa đơn (invoice):
 - `invoice_number`: Số hóa đơn.
 - `date`: Ngày hóa đơn.
 - `total`: Tổng tiền.
 - `vendor`: Thông tin nhà cung cấp (tên, VAT, địa chỉ).
 - `lines`: Chi tiết dòng hóa đơn (sản phẩm, số lượng, giá).
- Chi phí (expense):
 - `amount`: Số tiền chi phí.
 - `date`: Ngày chi phí.
 - `description`: Mô tả chi phí.
 - `category`: Danh mục chi phí.
- Sao kê ngân hàng (bank_statement):
 - `transactions`: Danh sách giao dịch (ngày, số tiền, mô tả).
 - `account_number`: Số tài khoản.

- *bank_name*: Tên ngân hàng.

e. Quy trình làm việc:

- Gửi tài liệu:
 - Sử dụng */odoo/extract/endpoint* để gửi PDF/hình ảnh.
 - Nhận *document_id* để theo dõi.
- Kiểm tra trạng thái:
 - Gọi */odoo/extract/status* để lấy trạng thái và dữ liệu trích xuất.
- Xác nhận và tích hợp:
 - Sử dụng */odoo/extract/validate* để lưu dữ liệu vào bản ghi Odoo.
- Xử lý lỗi:
 - Nếu trạng thái là *error*, kiểm tra thông báo lỗi và thử lại.

f. Tích hợp với mô hình Odoo:

Sau khi trích xuất, dữ liệu có thể được tích hợp vào các mô hình:

- **account.move**: Hóa đơn (bán/mua).
- **hr.expense**: Chi phí nhân viên.
- **account.bank.statement**: Sao kê ngân hàng.

2.3.3. Liên kết với External API

Extract API là một phần của **External API** trong Odoo, nhưng khác với **JSON-2 API** và **XML-RPC/JSON-RPC**:

- **JSON-2 API**: Dùng để gọi các phương thức ORM (như *create*, *search*) trên các mô hình.
- **Extract API**: Chuyên biệt cho xử lý tài liệu, không gọi trực tiếp phương thức ORM mà tích hợp dữ liệu trích xuất vào mô hình.
- **Tích hợp**: Có thể kết hợp Extract API với JSON-2 API để tự động hóa quy trình, ví dụ: trích xuất dữ liệu hóa đơn rồi tạo bản ghi *account.move* qua JSON-2 API.

2.3.4. Hướng dẫn thực hiện:

Để sử dụng Extract API, có thể làm theo các bước sau:

- Chuẩn bị:
 - Đăng ký tài khoản IAP và nạp Extract credits.
 - Tạo API key và lấy UUID cơ sở dữ liệu.
 - Chuẩn bị tài liệu (PDF hoặc hình ảnh).

- Gửi tài liệu:
 - Sử dụng công cụ như *requests* (Python) để gửi yêu cầu tới */odoo/extract/endpoint*.
 - Lưu *document_id* từ phản hồi.
- Kiểm tra trạng thái:
 - Gọi */odoo/extract/status* lặp lại (polling) cho đến khi trạng thái là *success* hoặc *error*.
 - Lấy dữ liệu trích xuất.
- Xác nhận và tích hợp:
 - Sử dụng */odoo/extract/validate* để lưu dữ liệu vào Odoo.
 - Hoặc dùng JSON-2 API để tạo bản ghi thủ công.
- Debugging:
 - Kiểm tra log trong Odoo (Settings → Technical → Logs).
 - Liên hệ hỗ trợ Odoo nếu hết credits hoặc gặp lỗi không rõ.

II. TÌM HIỂU WEBHOOK

1. Tổng quan về webhook

1.1. Khái niệm Webhook

Webhook là một cơ chế tự động giúp các ứng dụng hoặc hệ thống trao đổi dữ liệu với nhau theo thời gian thực (Real-time). Cơ chế này hoạt động dựa trên nguyên tắc "đẩy" (push): ngay khi có một sự kiện cụ thể xảy ra tại nguồn, dữ liệu sẽ được gửi đi mà không cần bên nhận phải gửi yêu cầu kiểm tra liên tục.

Ví dụ: Khi một đơn hàng được tạo thành công trên Website, Webhook sẽ tự động bắn thông báo về hệ thống ERP để tạo phiếu kho ngay lập tức.

1.2. Tại sao cần sử dụng Webhook?

Thay vì sử dụng cơ chế API truyền thống (Polling) bắt buộc hệ thống phải liên tục "hỏi" server xem có dữ liệu mới không (gây tốn tài nguyên và độ trễ), Webhook cho phép:

- **Cập nhật tức thì:** Hệ thống đích nhận được thông tin ngay khi sự kiện diễn ra.
- **Tối ưu tài nguyên:** Loại bỏ các request rỗng không cần thiết, giảm tải cho server.
- **Tự động hóa quy trình:** Dễ dàng tích hợp các dịch vụ như thanh toán, logistics, CRM, và CI/CD.

1.3. Quy trình hoạt động (4 Bước)

Cơ chế hoạt động cơ bản của Webhook bao gồm 4 bước chính:

1. **Cung cấp Endpoint:** Ứng dụng đích cung cấp một địa chỉ URL (Webhook Endpoint) để nhận dữ liệu.
2. **Đăng ký sự kiện (Trigger):** Ứng dụng nguồn cấu hình để theo dõi một sự kiện cụ thể (Ví dụ: Tạo đơn hàng, Khách hàng mới).
3. **Gửi HTTP Request:** Khi sự kiện xảy ra, ứng dụng nguồn đóng gói dữ liệu (Payload JSON/XML) và gửi qua phương thức HTTP POST đến Endpoint đã đăng ký.
4. **Xử lý và Phản hồi:** Ứng dụng đích nhận dữ liệu, xử lý (lưu trữ, kích hoạt tác vụ) và trả về mã trạng thái (ví dụ: 200 OK) để xác nhận.

1.4. Các thành phần kỹ thuật

- **Event Source:** Nguồn phát sinh sự kiện.

- **Webhook Endpoint:** URL đích để nhận dữ liệu.
- **Payload:** Gói dữ liệu chứa thông tin sự kiện (thường là JSON).
- **Authentication:** Cơ chế bảo mật (Token, Signature) để đảm bảo dữ liệu đến từ nguồn tin cậy.

2. Triển khai webhook trong odoo

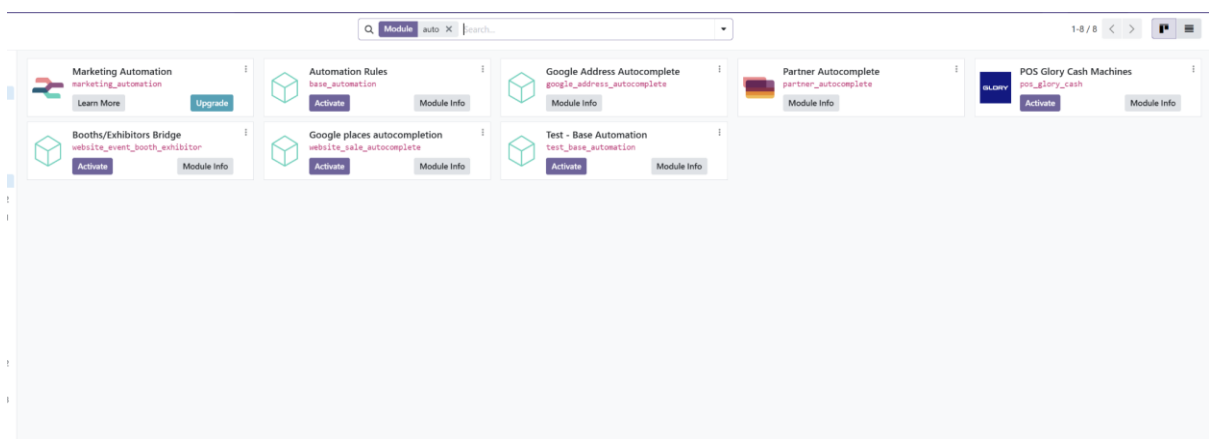
Trong Odoo, Webhook hoạt động dựa trên kiến trúc **Modular** và **Server Actions**. Có hai chiều hoạt động chính:

1. **Incoming Webhook:** Odoo nhận dữ liệu từ bên ngoài (thông qua [http.route](#) Controller).
2. **Outgoing Webhook:** Odoo gửi dữ liệu đi khi bản ghi thay đổi (thông qua [Automated Actions](#)).

Bước 1: Kích hoạt Module Automation Rules

Để Odoo có thể tự động theo dõi sự kiện, cần cài đặt module quy tắc tự động.

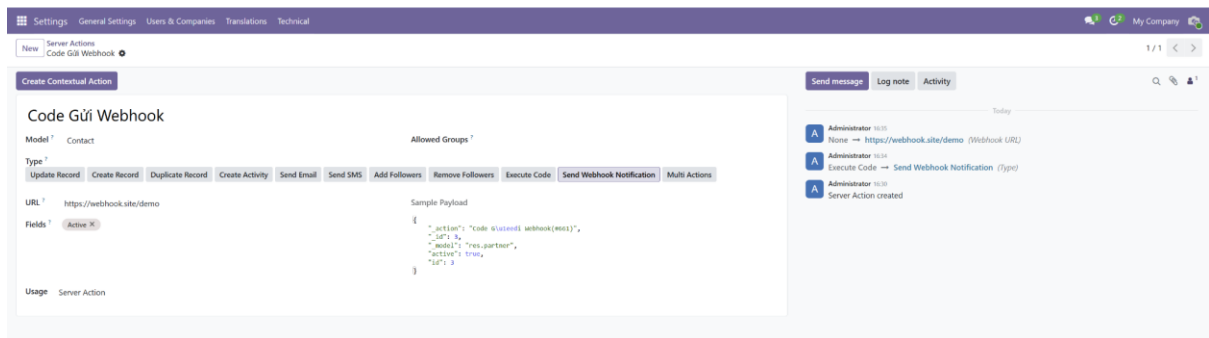
- Truy cập **Apps**, tìm kiếm từ khóa [automation](#).
- Kích hoạt module **Automation Rules** ([base_automation](#)).



Bước 2: Tạo Quy tắc tự động (Automation Rule)

Thiết lập "cò súng" (Trigger) để Odoo biết khi nào cần gửi Webhook.

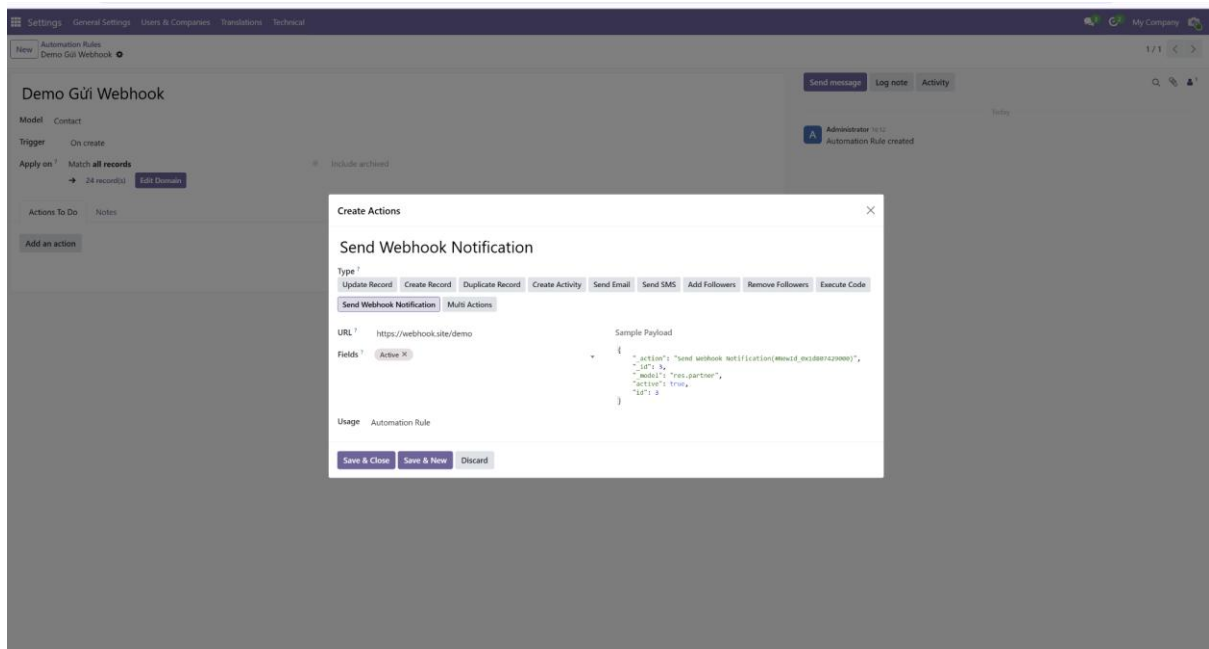
- Truy cập: Settings > Technical > Automation > Automation Rules.
- Tạo mới (New) và điền thông tin:
 - **Model:** Chọn đối tượng dữ liệu (Ví dụ: [Contact](#) - Liên hệ).
 - **Trigger:** Chọn sự kiện kích hoạt (Ví dụ: [On create](#) - Khi tạo mới).



Bước 3: Cấu hình hành động gửi Webhook (Action)

Trong các phiên bản Odoo mới, thay vì phải viết mã Python (dễ gặp lỗi bảo mật **forbidden opcode**), Odoo cung cấp sẵn tính năng **Send Webhook Notification**.

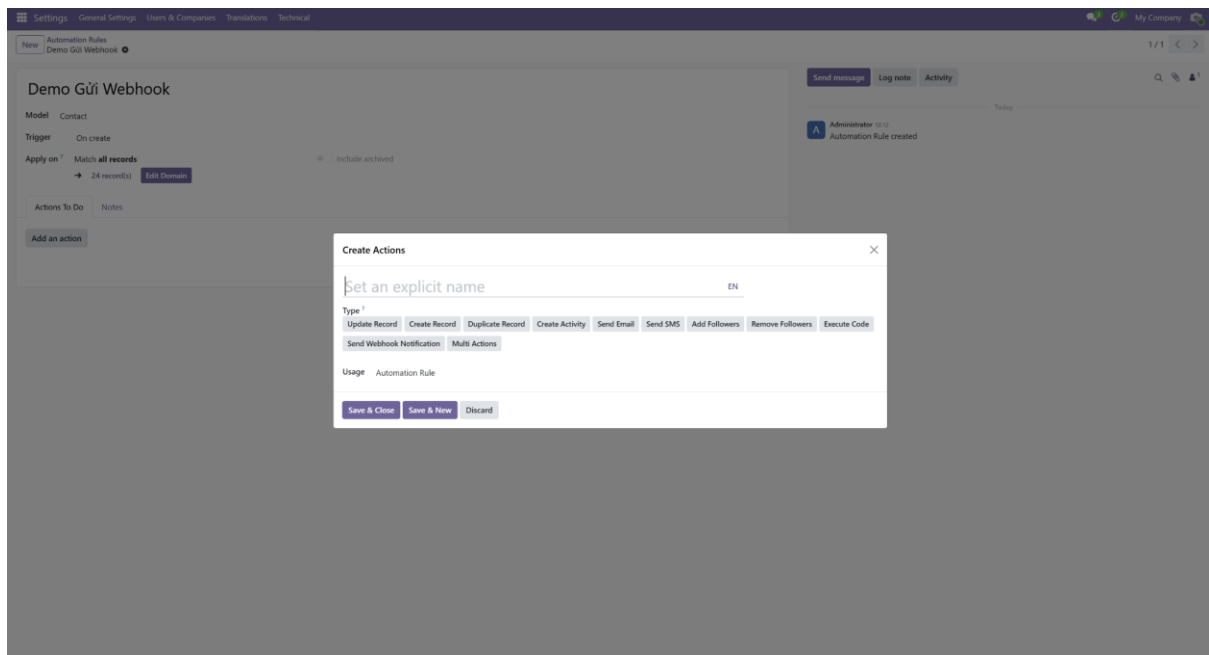
- Trong phần Actions To Do, chọn Add an action.
- Tại cửa sổ Create Actions, chọn loại: Send Webhook Notification.



Bước 4: Thiết lập Endpoint và Payload

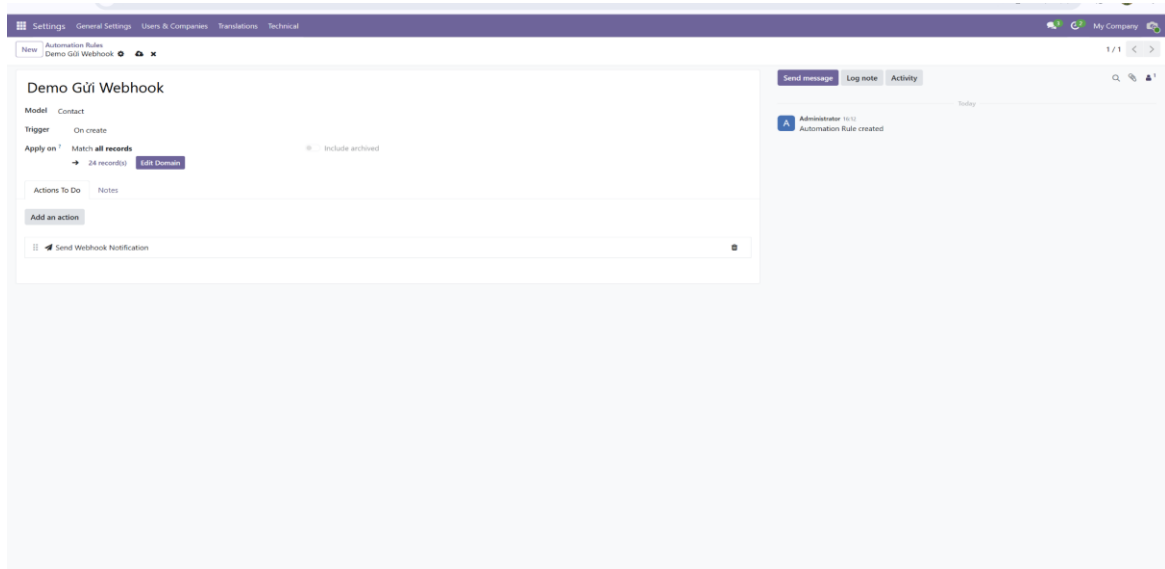
Tại giao diện cấu hình Webhook:

- **URL:** Nhập địa chỉ Endpoint của hệ thống nhận (Ví dụ: <https://webhook.site/...>).
- **Fields:** Chọn các trường dữ liệu muốn gửi đi (Ví dụ: **Name**, **Email**, **Phone**). Odoo sẽ tự động đóng gói các trường này thành JSON.



Bước 5: Hoàn tất và Kiểm thử

- Bấm **Save & Close** để lưu hành động.
- Bấm **Save** để lưu quy tắc tự động.
- **Kiểm thử:** Tạo một Contact mới trong Odoo. Hệ thống sẽ tự động kích hoạt hành động và gửi dữ liệu sang URL đã cấu hình.



III. VÍ DỤ

1. Tổng quan

Trong bối cảnh chuyển đổi số, dữ liệu khách hàng đóng vai trò là tài sản chiến lược của doanh nghiệp. Tuy nhiên, thực trạng phân tán dữ liệu trên đa nền tảng – từ website, mạng xã hội (Zalo, Facebook) đến các sàn thương mại điện tử – đang đặt ra thách thức lớn trong công tác quản trị. Yêu cầu cấp thiết đối với các nhà quản lý công nghệ là thiết lập quy trình đồng bộ hóa dòng chảy dữ liệu về một "nguồn sự thật duy nhất" (Single Source of Truth), cụ thể là hệ thống Quản trị Quan hệ Khách hàng (CRM).

Odoo, với vai trò là nền tảng ERP mã nguồn mở, cung cấp khả năng tích hợp mở rộng thông qua External API. Khác với kiến trúc RESTful thịnh hành thấy ở các nền tảng SaaS như Salesforce hay HubSpot, Odoo vận hành dựa trên giao thức Remote Procedure Call (RPC). Phần nội dung dưới đây sẽ tập trung phân tích kỹ thuật tích hợp giữa PHP và Odoo thông qua JSON-RPC, đồng thời làm rõ các giải pháp tối ưu hiệu năng và xử lý dữ liệu đặc thù tại thị trường Việt Nam.

2. Cơ sở lý luận: Kiến trúc RPC và xu hướng chuyển dịch sang JSON

2.1. Đặc điểm kiến trúc API của Odoo

Kiến trúc API của Odoo không thiết lập các điểm cuối (endpoint) riêng biệt cho từng đối tượng dữ liệu (như `/api/leads` hay `/api/orders`). Thay vào đó, hệ thống cung cấp một cổng giao tiếp chung, cho phép các ứng dụng bên thứ ba gọi trực tiếp các phương thức (method) được định nghĩa trong mã nguồn Python.

Yếu tố cốt lõi tạo nên sự linh hoạt này là hàm `execute_kw`. Đây là một hàm "meta" cho phép thực thi bất kỳ phương thức nào của mô hình dữ liệu (Model), với điều kiện người dùng được cấp quyền truy cập tương ứng.

Công thức tổng quát được mô tả như sau:

`execute_kw(db, uid, password, model, method, args, kwargs)`

Cơ chế này cho phép các nhà phát triển tận dụng ngay lập tức các trường tùy chỉnh (custom field) mà không cần tái cấu trúc API, một ưu điểm quan trọng trong các dự án phát triển theo mô hình Agile.

2.2. Xu hướng thay thế XML-RPC bằng JSON-RPC

Trước đây, XML-RPC được xem là chuẩn mực mặc định. Tuy nhiên, các báo cáo kỹ thuật gần đây chỉ ra rằng từ phiên bản PHP 8.0, các tiện ích mở rộng XML-RPC đã không còn được kích hoạt mặc định, gây trở ngại cho công tác bảo trì hệ thống. Bên cạnh đó, lộ trình phát triển của Odoo (dự kiến trong các phiên bản 19/20) cũng cho thấy xu hướng loại bỏ dần các giao thức cũ để chuyển sang kiến trúc hiện đại hơn.

Trong bối cảnh đó, JSON-RPC (JavaScript Object Notation RPC) nổi lên như giải pháp thay thế tối ưu nhờ các đặc tính: dung lượng nhẹ (lightweight), cấu trúc dễ đọc và khả năng tương thích cao với hệ sinh thái web/mobile hiện đại (ReactJS, VueJS, Mobile Apps).

3. Phân tích ví dụ thực tiễn: Tích hợp Web Form PHP vào Odoo CRM

Trong phạm vi nghiên cứu, đề án tập trung phân tích kịch bản triển khai thực tế tại các doanh nghiệp Việt Nam: Tự động hóa quy trình đồng bộ dữ liệu khách hàng từ Website (nền tảng PHP/WordPress) nhằm khởi tạo các bản ghi Tiềm năng (Lead) trong hệ thống Odoo CRM.

3.1. Quy trình kỹ thuật

Quy trình tích hợp thông qua JSON-RPC bao gồm 3 bước chính:

1. **Xác thực (Authentication):** Hệ thống PHP gửi yêu cầu đến endpoint `/jsonrpc` để xác thực người dùng. Về mặt bảo mật, việc sử dụng **API Key** (được tạo trong *Preferences > Account Security*) là bắt buộc thay cho mật khẩu gốc, nhằm ngăn chặn rủi ro lộ thông tin đăng nhập của tài khoản quản trị.
2. **Chuẩn bị Payload (Gói tin):** Dữ liệu cần được đóng gói tuân thủ chuẩn JSON-RPC 2.0.
3. **Thực thi (Execution):** Gọi phương thức `create` trên đối tượng `crm.lead`.

3.2. Giải quyết vấn đề bản địa hóa (Localization) và bảng mã ký tự

Một trong những vấn đề kỹ thuật trọng yếu khi triển khai tại thị trường Việt Nam là xử lý bảng mã ký tự (Encoding). Các hệ thống cũ hoặc cấu hình không chuẩn xác thường gửi dữ liệu dưới dạng Latin1. Điều này dẫn đến tình trạng các dữ liệu tiếng Việt (ví dụ: tên khách hàng "Nguyễn Văn An") bị lỗi font (ký tự rác) khi lưu trữ vào Odoo, ảnh hưởng trực tiếp đến tính toàn vẹn dữ liệu và khả năng truy xuất thông tin sau này.

Giải pháp kỹ thuật bắt buộc trong mã nguồn PHP bao gồm:

- Khai báo header: `Content-Type: application/json; charset=utf-8`.

- Sử dụng tham số **JSON_UNESCAPED_UNICODE** trong quá trình mã hóa dữ liệu JSON.

Đoạn mã xử lý tiêu biểu:

```
$payload = json_encode($request_data, JSON_UNESCAPED_UNICODE);  
curl_setopt($ch, CURLOPT_HTTPHEADER, array(  
    'Content-Type: application/json; charset=utf-8',  
    'Content-Length: ' . strlen($payload)  
));
```

Việc tuân thủ nghiêm ngặt chuẩn Unicode đảm bảo dữ liệu được hiển thị chính xác và đồng nhất trong môi trường đa ngôn ngữ.

4. Tối ưu hóa hiệu năng và bảo mật hệ thống

4.1. Ứng dụng kỹ thuật Xử lý theo lô (Batch Processing)

Một hạn chế thường gặp trong quá trình phát triển là việc thực hiện gọi API tuần tự trong vòng lặp (For loop). Ví dụ, để khởi tạo 1.000 leads, việc gọi hàm **create** 1.000 lần sẽ tạo ra số lượng kết nối HTTP tương ứng, gây tắc nghẽn băng thông và quá tải cơ sở dữ liệu.

Odoo ORM hỗ trợ cơ chế xử lý theo lô hiệu quả. Thay vì gửi từng bản ghi đơn lẻ, hệ thống cho phép gửi một danh sách (list) chứa nhiều đối tượng dữ liệu. Odoo sẽ chuyển đổi danh sách này thành một câu lệnh SQL **INSERT INTO ... VALUES ...** duy nhất, giúp cải thiện hiệu suất xử lý gấp nhiều lần so với phương pháp tuần tự.

4.2. Cơ chế kiểm soát truy cập (ACL)

Để đảm bảo an toàn thông tin, nguyên tắc "Quyền tối thiểu" (Least Privilege) cần được áp dụng triệt để. Tài khoản sử dụng cho API không nên được cấp quyền Quản trị (Settings/Administration). Thay vào đó, cần thiết lập một tài khoản chuyên biệt (ví dụ: **api_user**) với quyền hạn giới hạn (ví dụ: *Sales / User: Own Documents Only*). Điều này giảm thiểu rủi ro truy cập trái phép vào dữ liệu nhạy cảm trong trường hợp khóa API bị xâm phạm.

5. Kết luận

Kiến trúc API của Odoo, đặc biệt là sự chuyển dịch sang giao thức JSON-RPC, thể hiện tính linh hoạt và khả năng thích ứng cao với công nghệ hiện đại. Qua phân tích mô hình tích hợp Web Form PHP, có thể thấy hiệu quả triển khai Odoo CRM tại Việt Nam được quyết định bởi ba yếu tố then chốt:

1. **Am hiểu giao thức:** Nắm vững cơ chế **execute_kw** và chuẩn JSON-RPC.
2. **Xử lý dữ liệu chính xác:** Tuân thủ chuẩn mã hóa UTF-8 đối với dữ liệu tiếng Việt.
3. **Tối ưu hóa hệ thống:** Áp dụng kỹ thuật Batch Processing để đảm bảo hiệu năng vận hành.

Đối với công tác quản trị và phát triển hệ thống, việc chủ động cập nhật các chuẩn kết nối mới và xây dựng các lớp trung gian (middleware) để xử lý các yêu cầu đặc thù địa phương (như Zalo, SMS Brandname) là yêu cầu tiên quyết để khai thác tối đa tiềm năng của Odoo CRM.

TÀI LIỆU THAM KHẢO

1. External JSON-2 API — Odoo 19.0 documentation
2. External RPC API — Odoo 19.0 documentation
3. Extract API — Odoo 19.0 documentation
4. ORM API — Odoo 19.0 documentation

